The **df** command in Ubuntu is used to display information about the disk space usage of the file system. Here's an example of the basic usage of the **df** command

```
df
```

This will display the disk space usage of all mounted file systems in the current directory. The output will include the total size of the file system, the amount of space used, the amount of free space, and the percentage of space used.

Here are some common options that can be used with the **df** command:

- **-h** or **--human-readable**: Displays sizes in a human-readable format, such as "1K", "1M", or "1G", instead of bytes.

  Example: **df -h**

- **-i** or **--inodes**: Displays information about the number of inodes used and available on the file system.

  Example: **df -i**

- **-T** or **--type**: Displays information about the file system type.

  Example: **df -T**

The `du` command in Ubuntu is used to estimate the amount of disk space used by a particular file or directory. The name `du` stands for "disk usage".

```
du filename
```

his will display the amount of disk space used by the specified file. If you want to see the disk space used by a directory and all its subdirectories, you can use the `-s` (or `--summarize`) option:

```
du -s directoryname
```

The `du` command will then display the total amount of disk space used by the specified directory, along with the amount of disk space used by each subdirectory.

Here are some other common options that can be used with the `du` command:

- `-h` or `--human-readable`: Displays sizes in a human-readable format, such as "1K", "1M", or "1G", instead of bytes.

The **`find`** command in Ubuntu is used to search for files and directories within a specified directory hierarchy. It can search for files based on a variety of criteria, such as name, type, size, and modification time.

```
find /home/user/documents -name "*.txt"
```

This will search for all files with the extension ".txt" within the /home/user/documents directory and its subdirectories.

Here are some other common options that can be used with the **`find`** command:

- **`-type`**: Searches for files of a specific type.
  Example: **`find /home/user -type d -name "dirname"`** searches for a directory named "dirname" within /home/user directory and its subdirectories.

- **`-size`**: Searches for files of a specific size.

  Example: **`find /home/user -size +10M`** searches for files that are larger than 10 megabytes.

- **`-mtime`**: Searches for files that were modified within a specific time frame.

  Example: **`find /home/user -mtime -7`** searches for files that were modified within the last 7 days.

- **`-user`** or **`-group`**: Searches for files owned by a specific user or group.

  Example: **`find /home/user -user johndoe`** searches for files owned by the user "johndoe".

The `locate` command in Ubuntu is used to search for files and directories on the entire file system quickly. It is much faster than the `find` command because it uses a pre-built database of the file system.

Here's an example of the basic usage of the `locate` command:

```
locate filename
```

By default, the `locate` command searches for files that have been modified within the last day. However, you can force the database to update using command

```
sudo updatedb
```

This command will update the database that `locate` uses to search for files and directories.

- `-i` or `--ignore-case`: Searches for files and directories regardless of their case.

  Example: `locate -i filename`

- `-l` or `--limit`: Limits the number of results displayed.

  Example: `locate -l 10 filename`

The `dd` command in Ubuntu is used to copy and convert data between files and devices. It is often used for creating bootable USB drives, making disk images, and backing up data.

The basic syntax of the `dd` command is as follows:

```
dd if=input_file of=output_file [options]
```

Here, `if` specifies the input file, and `of` specifies the output file.

Here are some common options that can be used with the `dd` command:

- `bs` or `blocksize`: Specifies the block size to use for reading and writing data.

  Example: `dd if=input_file of=output_file bs=4k`

- `count`: Specifies the number of blocks to copy.

  Example: `dd if=input_file of=output_file count=10`

- `status`: Displays the progress of the copy operation.

  Example: `dd if=input_file of=output_file status=progress`

Example :

```
dd if=input_file.txt of=output_file.txt
```

his command copies the contents of `input_file.txt` to `output_file.txt`. The `if` option specifies the input file (`input_file.txt`), and the `of` option specifies the output file (`output_file.txt`).

Note that if the output file already exists, `dd` will overwrite its contents without warning. Always double-check your input and output files before running any `dd` command to avoid data loss.

```
dd if=ubuntu-20.04.2.0-desktop-amd64.iso of=/dev/sdb bs=4M status=progress
```

This command creates a bootable USB drive using the Ubuntu 20.04 ISO image. The `if` option specifies the input file (`ubuntu-20.04.2.0-desktop-amd64.iso`), and the `of` option specifies the output file or device (`/dev/sdb`). The `bs` option specifies the block size to use for reading and writing data, and the `status` option displays the progress of the copy operation.

To backup the entire harddisk : To backup an entire copy of a hard disk to another hard disk connected to the same system, execute the dd command as shown. In this dd command example, the UNIX device name of the source hard disk is /dev/hda, and device name of the target hard disk is /dev/hdb.

#dd if = /dev/sda of = /dev/sdb

where,

- **if=/dev/sda** : Input disk (source)
- **of=/dev/sdb** : Output disk (destination)

Need of backup:

1. Data Loss Prevention

We‘ve all heard about or experienced a tragic loss of data. The main reason for data backup is to save important files if a system crash or hard drive failure occurs.

2. Operation Plan B

There should be additional data backups if the original backups result in data corruption or hard drive failure. This option is best done via the cloud or offsite storage. Additional backups are necessary if natural or man-made disasters occur. Storms and warfare can lead to the destruction of servers and computers due to fires and floods. Luckily, we are in the age of cloud technology, where backup your data has become easier and more secure than ever before.

3. Tax Reporting and Audits

Tax authorities are notorious for audits. Laws differ among countries, but it is important for companies to save financial and accounting data for tax reporting purposes. With data backup, companies can save face during audits.

4. Client Relationship

Saved information improves client relationship management, which leads to increased marketing and sales. Additionally, saved client information builds trust and value of a company.

5. Investor Relations

Per investor relations, data backup reduces the tedious time to compile annual reports to shareholders. Saved information symbolizes a company's due diligence and organization. Without data backup, shareholders cannot make informed decisions or determine a company's value.

6. Archiving

Backed up information streamlines the development of archives. With digital information, company history is in the making.

7. Competitive Gain

Saved company data can be a competitive advantage because there are many businesses that fail to data backup important information.

8. Improved Productivity

With existing backed up files, companies improve productivity by reducing wasted time. Archived files lead to comparative studies of the past and present to devise a more effective plan.

9. No Wasted Time

Data backup reduces _wasted time‘ by preventing repetitions. Thus, employees do not have to rewrite reports.

10. Peace of Mind

Regular data backups lead to peace of mind. In the event, a cybercrime, system crashes or disasters occur, there is a backup ready to go to restart a company's archive. It is never too late to start saving important company data. In the end, data backup is necessary to save the business from losing investors and customers and closing down.

he **tar** command in Ubuntu is used to create and extract tar archives. Tar (short for "tape archive") is a standard format for storing files and directories in a single file. The basic syntax of the **tar** command is as follows:

```
tar [options] [file or directory]
```

Here are some common options that can be used with the **tar** command:

- **-c** or **--create**: Creates an archive.

  Example: **tar -cvf archive.tar file1 file2 dir1**

  This command creates a new archive named **archive.tar** and adds **file1**, **file2**, and **dir1** to it. The **-c** option specifies that **tar** should create an archive, and **-v** specifies verbose output(display more detailed information).

- **-x** or **--extract**: Extracts files from an archive.

  Example: **tar -xvf archive.tar**

  This command extracts the contents of the **archive.tar** file to the current directory. The **-x** option specifies that **tar** should extract files from the archive.

- **-t** or **--list**: Lists the contents of an archive.

  Example: **tar -tvf archive.tar**

  This command lists the contents of the **archive.tar** file. The **-t** option specifies that **tar** should list the contents of the archive.

The `cpio` command in Ubuntu is used to create and extract archives in the `cpio` format. `cpio` (short for "copy in, copy out") is a command-line utility that can copy files to and from an archive file or device. It is similar to the `tar` command, but uses a different format for storing files and directories. The basic syntax of the `cpio` command is as follows:

```
cpio [options] < file_list
```

Here, `file_list` is a list of files and directories to include in the archive.

Here are some common options that can be used with the `cpio` command:

- `-i` or `--extract`: Extracts files from an archive.

  Example: `cpio -i < archive.cpio`

- `-o` or `--create`: Creates an archive.

  Example: `find . | cpio -o > archive.cpio`

- `-v` or `--verbose`: Displays verbose output.

`cpio` and `tar` are both used for creating and extracting archives of files, but they use different formats. `cpio` archives are organized as a sequence of file records, while `tar` archives are organized as a single stream of bytes.

One advantage of the `cpio` format is that it allows for more flexible handling of filenames and paths, particularly for files with unusual or non-printable characters in their names. The `cpio` format also supports the preservation of hard links and special file attributes, which can be important for preserving the functionality of certain applications or systems.

In summary, while `cpio` has some unique features that make it useful in certain situations, `tar` remains the more commonly used and versatile archiving tool in Ubuntu and other Linux systems.

The `fdisk` command is a Linux utility that is used for partitioning hard disks. It allows you to create, delete, and modify partitions on a hard drive.

To use the `fdisk` command, you must first identify the device that you want to partition. This can be done using the `lsblk` command. Once you have identified the device, you can run `fdisk` with the appropriate options.

Here are some common options that can be used with the `fdisk` command:

- `-l` or `--list`: Lists all available disks and their partitions.

  Example: `fdisk -l`

  This command lists all available disks and their partitions.

- `-n` or `--new`: Creates a new partition.

  Example: `fdisk /dev/sdb`

  This command launches `fdisk` with the `/dev/sdb` device and allows you to create a new partition. You will be prompted to enter the partition size and other information.

- `-d` or `--delete`: Deletes a partition.

  Example: `fdisk /dev/sdb`

  This command launches `fdisk` with the `/dev/sdb` device and allows you to delete a partition. You will be prompted to enter the partition number to delete.

- `-p` or `--print`: Prints the partition table.

  Example: `fdisk -l /dev/sdb`

  This command lists the partition table for the `/dev/sdb` device.

`pydf` is a command-line utility in Linux that provides a more visually appealing and user-friendly way to display disk usage statistics than the standard **`df`** command.

`pydf` presents disk usage information in a hierarchical format, with disk usage statistics for each mounted filesystem shown in a separate column. This makes it easier to understand the disk usage of each filesystem, especially when multiple filesystems are involved.

`pydf` also shows additional information such as the percentage of disk usage, the amount of available space, and the type of filesystem. Additionally, it supports color-coded output to highlight different levels of disk usage.

```
pydf
```

This command displays disk usage information for all mounted filesystems.

```
pydf /dev/sda1
```

This command displays disk usage information for the **`/dev/sda1`** filesystem.

```
pydf -h
```

This command displays disk usage information in human-readable format, showing sizes in units such as KB, MB, GB, etc.

`lsblk` is a Linux command-line utility that lists information about all available block devices (such as hard disks, USB drives, and CD/DVD drives) and their corresponding attributes such as size, partition information, file system type, mount point, and more.

Here are some examples of how to use **`lsblk`**:

```
Lsblk
```

This command displays all block devices on the system, along with their corresponding attributes.

```
lsblk -t
```

This command displays block devices in a tree format, showing the parent-child relationships between block devices.

```
lsblk -f ext4
```

This command displays all block devices that have the ext4 file system.

```
lsblk /dev/sda1
```

This command displays the block device **`/dev/sda1`** and its attributes.

`parted` is a Linux command-line utility used to create, resize, and manipulate disk partitions. It allows you to create new partitions, resize or delete existing partitions, and modify partition properties such as the file system type and partition label.

```
sudo parted /dev/sda print
```

This command displays the partition table of the **`/dev/sda`** disk.

```
sudo parted /dev/sda mkpart primary ext4 0% 100%
```

This command creates a new primary partition on the **`/dev/sda`** disk with an ext4 file system, spanning the entire disk.

```
sudo parted /dev/sda resizepart 1 10GB
```

This command resizes the first partition on the **`/dev/sda`** disk to 10GB.

```
sudo parted /dev/sda set 1 ext3
```

This command sets the file system type of the first partition on the **`/dev/sda`** disk to ext3.

`Note: parted` **can be a powerful tool for managing disk partitions, but it also carries some risks. Any changes made to the partition table using** **`parted`** **can result in data loss, so it is important to be cautious and double-check your commands before running them.**

---

sfdisk is a Linux command-line utility used to manage partition tables on a disk. It allows you to create, modify, and delete partitions on a disk in a scriptable way. It can be used to create multiple partitions on a disk, change the partition type, set the partition size, and more.
Unlike other partitioning tools, **sfdisk** operates on partition tables directly, rather than on the block devices themselves. This makes it possible to perform operations on disks even when they are in use.

```
sudo sfdisk -l /dev/sda
```

This command displays the partition table of the **/dev/sda** disk.

```
sudo sfdisk -l /dev/sda
```

This command displays the partition table of the **/dev/sda** disk.

```
echo ",,L" | sudo sfdisk /dev/sda
```

This command creates a new partition on the **/dev/sda** disk, with no specific size or file system type.

In the **echo ",,L" | sudo sfdisk /dev/sda** command, **,,L** is the argument passed to **sfdisk** to create a new partition.

Here's what each of the characters means:

- The first comma **,** specifies the starting sector of the partition (which is left blank to let **sfdisk** automatically select the next available sector).
- The second comma **,** specifies the ending sector of the partition (which is also left blank to let **sfdisk** automatically select the end of the disk).
- The **L** specifies the partition type as "Linux".

L partition type specifies that the partition will be formatted with a Linux file system, such as ext4 or XFS, which are commonly used on Linux systems.

```
sudo sfdisk --delete /dev/sda 2
```

This command deletes the second partition on the **/dev/sda** disk.

`cfdisk` is a Linux command-line utility used for partitioning a disk with a simple and easy-to-use interface. It is similar to other partitioning tools such as **`fdisk`** and **`parted`**, but with a more user-friendly interface.

When you run **`cfdisk`**, it displays a menu-driven interface that allows you to view, create, modify, and delete partitions on a disk. The interface is divided into three sections:

1. A list of partitions on the disk, showing their type, size, and file system.
2. A partition editing menu, where you can create, modify, and delete partitions.
3. A command menu, where you can perform other operations such as saving or quitting.

```
sudo cfdisk /dev/sda
```

This command launches **`cfdisk`** and displays the partition table of the **`/dev/sda`** disk.

```
sudo cfdisk /dev/sda
```

This command launches **`cfdisk`**, where you can select the **`New`** option from the partition editing menu to create a new partition.

```
sudo cfdisk /dev/sda
```

his command launches **`cfdisk`**, where you can select the partition you want to delete and then choose the **`Delete`** option from the partition editing menu.

`cfdisk` is a simple and easy-to-use partitioning tool that is useful for creating and modifying partitions on Linux systems. However, it is not as powerful or flexible as other partitioning tools such as **`parted`**, which provides more advanced features such as resizing partitions and supporting more partition table types.